Python Engineering - Reference Guide

CLASS 14 - File I/O | Syntax, functions and paths

This guide is intentionally detailed. It is designed to remain useful months later, explaining not only syntax but what each operation means and why we use it.

1. open()

What it does: asks the operating system for access to a file and returns a **file object** through which Python can read or write.

Important inputs: path, mode, and optional encoding.

Returns: a file object.

```
file = open(
    'data/results.txt',
    'r',
    encoding='utf-8'
)
```

Opening a file does not necessarily load all contents. It creates an interface to the resource.

2. File modes

Mode	Meaning	If missing	If already exists
'r'	Read	Error	Reads without deleting
'w'	Write	Creates it	Replaces contents
'a'	Append	Creates it	Keeps and appends
'x'	Exclusive creation	Creates it	Error
'b'	Binary modifier	-	Combine: 'rb', 'wb'

3. with open(...) as file

```
with open(file_path, 'r', encoding='utf-8') as file:
    data = file.read()

# file is closed here
```

`with` uses a context manager so the resource is closed reliably.

4. file.write()

Receives: text string. **Returns:** number of characters written.

```
value = 3.14159
file.write(f'{value:.3f}\n')
```

For a text file, numerical values must be represented as text.

5. Newline \n

```
file.write('first line\n')
file.write('second line\n')
```

6. read(), readline(), readlines(), iteration

Operation	Returns	Typical use
file.read()	Remaining content as one string	Small files / read all
file.readline()	One string line	Consume header/one line
file.readlines()	List of strings	Load all lines
for line in file	One line per iteration	Efficient line processing

7. strip()

```
line = '100.0,20.0,0.0\n'
clean = line.strip()
# '100.0,20.0,0.0'
```

Returns a new string with edge whitespace/newlines removed.

8. split(separator)

```
values = '100.0,20.0,0.0'.split(',')
# ['100.0', '20.0', '0.0']
```

The pieces are still strings.

9. float()

```
value = float('20.0')
# numeric 20.0
```

10. encoding='utf-8'

Text files store bytes. Encoding defines how characters map between Python text and bytes. Explicit UTF-8 makes the behavior more reproducible.

11. pathlib.Path

```
from pathlib import Path
path = Path('data') / 'vertical_trajectory.csv'
```

Path represents a path as an object. Its / operator joins path components.

12. __file__

```
base_dir = Path(__file__).resolve().parent
```

Expression	Meaning
Path(__file__)	Convert script path to Path
.resolve()	Absolute normalized path
.parent	Directory containing the file

13. Relative vs absolute paths

```
# relative
Path('data/results.csv')

# absolute example
Path('/home/user/project/data/results.csv')
```

Relative paths need a reference point. In VS Code that often depends on the current working directory, not the script directory. Anchoring to `__file__` avoids that common confusion.

14. Path.mkdir()

```
data_dir.mkdir(exist_ok=True)
```

`exist_ok=True`: if the folder already exists, do not treat that as an error.

15. CSV

CSV is a **text format**, not a NumPy matrix or an Excel workbook.

```
h_m,v_m_s,t_s
100.000000,20.000000,0.000000
```

It is simple and portable, but rich data types are not preserved automatically.

16. np.array(rows)

```
rows = [[100.0,20.0,0.0], [102.0,19.019,0.1]]
trajectory = np.array(rows)
```

17. np.allclose()

```
np.allclose(original, loaded)
```

Appropriate for floating-point data, especially after decimal text formatting.

18. Round-trip validation

```
numbers -> write/serialize -> file
        -> read/parse -> recovered numbers -> compare
```

19. Module responsibilities

Module	Responsibility
vertical_state.py	Physical model/state and behavior
file_io.py	Python data <-> file representation
main.py	Application orchestration
data/	Persistent generated outputs

This is **separation of concerns**.

20. Final cheat sheet

```
open(path, 'r')  -> read
open(path, 'w')  -> write/replace
open(path, 'a')  -> append

with open(...) as file: -> automatic close
file.write(str)          -> write text
file.readline()          -> one line
line.strip()             -> trim edges/newline
line.split(',')          -> list of strings
float(text)              -> text to float

Path(__file__).resolve().parent -> script directory
data_dir.mkdir(exist_ok=True)   -> ensure folder
np.allclose(a, b)               -> approximate check
```